

NexusTalk

Software Design Document

Version 1.0 - October 10, 2025

Document Control

Designers: Arav Sawant & Rishabh Raj
Subject: Software and Architecture design of NexusTalk
Status: Draft for review
Audience: Stakeholders, Developers, QA, Security, Product Managers

Contents

1	Introduction & Design Goals	1
2	Architecture Overview (4+1 Views)	2
2.1	Architectural Style	2
2.2	System Context	2
2.3	4+1 Views Summary	2
2.3.1	Logical View	2
2.3.2	Process View	2
2.3.3	Development View	2
2.3.4	Deployment View	3
2.3.5	Use-Case View	3
3	Logical View	4
3.1	Layering	4
3.2	Service Catalogue	4
3.3	Data Ownership	4
4	Process View	5
4.1	Key Runtime Components	5
4.2	Representative Flows	5
4.2.1	Send E2EE Message	5
4.2.2	File Sharing	5
4.2.3	Channel Scheduled Post	5
4.2.4	Voice/Video Call	5
4.2.5	Report Content	5
5	Deployment/Implementation View	6
5.1	Topology	6
5.2	Stores	6
5.3	Scalability & Availability	6
6	Development View	7
6.1	Repository Layout	7
6.2	Practices	7
7	Interface Specifications	8
7.1	Authentication	8
7.2	Messaging	8
7.3	Groups & Channels	8
7.4	Moderation	8

7.5	Payments	8
8	Feature Designs (by SRS Modules)	9
8.1	Core Communication	9
8.2	Multi-Device	9
8.3	Privacy Tools	9
8.4	Channels & Communities	9
8.5	AI Features	9
8.6	Moderation & Safety	9
9	Data Design & Schema	10
10	Security & Privacy	11
10.1	End-to-End Encryption	11
10.2	Key Management	11
10.3	Privacy Controls	11
10.4	Hardening	11
11	Quality Attributes & NFR Mapping	12
12	Sequence Models	13
12.1	Text Message Send	13
12.2	Channel Scheduled Post	13
12.3	Call Setup	13
13	State Models	14
13.1	Message Lifecycle	14
13.2	Call Lifecycle	14
13.3	Report Ticket	14
14	Risks, Testing & Traceability	15
14.1	Key Risks	15
14.2	Testing Strategy	15
14.3	SRS–Design Traceability	15

CHAPTER 1

Introduction & Design Goals

Purpose. This document transforms the NexusTalk SRS into a concrete, implementable design. It presents a complete architecture and detailed component design aligned with the SRS scope and constraints: cross-platform messaging; end-to-end encryption (Signal Protocol) for 1:1 and private groups; scalable microservices; on-device privacy features; AI-assisted capabilities; channels, communities, moderation, and payments.

Method. We structure the SDD using the two-stage design approach—high-level (architectural) followed by detailed design—per standard guidance. We adopt layered and client-server architectures, MVC for the clients, and provide multiple architectural *views* (4+1), sequence and state models, and interface specifications, mirroring recommended practice.

Design Principles. The design aims for high cohesion and low coupling, clean layering, abstraction, reusability, flexibility, portability, testability, and defensive design. These principles are embedded in module boundaries, APIs, and deployment.

CHAPTER 2

Architecture Overview (4+1 Views)

2.1 Architectural Style

NexusTalk uses a **layered, microservices** architecture deployed in the cloud and accessed by clients (Android, iOS, Web) over the Internet. The UI layer follows **Model-View-Controller (MVC)** on each client. Services communicate primarily via synchronous REST/gRPC through an API Gateway and asynchronously via an event bus (Kafka). Media and RTC paths use dedicated services (upload service, CDN, TURN/S-TUN, and SFU for group calls).

2.2 System Context

- **Actors:** End users, channel owners/admins, moderators, and payment processors.
- **External systems:** APNS/FCM push, Payment gateway, Cloud object storage/CDN, LLM/STT providers.

2.3 4+1 Views Summary

2.3.1 Logical View

Core domains are decomposed into services: Identity, Session/Device, Messaging, Group & Channels, Presence, Calls/RTC, Media, Search, Payments, Moderation, Analytics, Notification, and Settings.

2.3.2 Process View

At runtime: clients connect to Gateway; stateless services scale horizontally; a Message Broker handles fan-out and persistence; Calls traverse TURN/STUN and SFU; background workers do scheduling, transcription, and summarization.

2.3.3 Development View

Codebase is a polyrepo or monorepo with per-service folders, shared libraries for common types and auth, API definitions (OpenAPI/proto), and CI/CD pipelines.

2.3.4 Deployment View

Kubernetes across multiple regions; databases with replication; object storage for media; a CDN; secrets management; observability stack (logs/metrics/traces).

2.3.5 Use-Case View

Representative scenarios: sending a message, voice/video call, creating a channel, scheduling a post, AI transcription, reporting abuse, payments.

CHAPTER 3

Logical View

3.1 Layering

Presentation (Clients) → Edge (Gateway) → Application Services → Data/Infrastructure. Higher layers depend on lower layers only.

3.2 Service Catalogue

Service	Responsibility
Identity & Auth	Registration/login by phone & OTP; user profile; RBAC; OAuth tokens.
Session & Devices	Device linking (multi-device), key provisioning, pre-keys, and session metadata.
Messaging	1:1 and group messaging, message states (sent, delivered, read), ephemeral timers, broadcasts.
Groups & Channels	Group lifecycle, roles/permissions, threads, community folders, channel scheduling.
Presence	Online/last-seen, typing, stealth mode rules.
Media	Upload/download, thumbnails, view-once, download-blocking, content-hashing.
Calls/RTC	Signalling, WebRTC, TURN/STUN, SFU for multi-party, recording (consent-gated).
Search	Full-text over messages (server), on-device media indexing (object/OCR).
Payments	In-chat transfers; channel subscriptions; ledger & gateway integration.
Moderation	Reporting, context capture, triage, automated sanctions/strikes.
Analytics	Channel analytics, dashboards, counters.
Notification	Push notifications, rate limiting.
Settings	Privacy matrices, username & discoverability.

3.3 Data Ownership

Each service owns its data (schema per service). Cross-service reads through APIs; cross-service writes via events to avoid tight coupling.

CHAPTER 4

Process View

4.1 Key Runtime Components

- **API Gateway** terminates TLS, authenticates, routes to services.
- **Message Broker** (Kafka) for fan-out, receipts, analytics, scheduled posts.
- **RTC Stack** (Signaling service + TURN/STUN + SFU) for voice/video calls.
- **Background Workers** for media processing, transcription, summarization.

4.2 Representative Flows

4.2.1 Send E2EE Message

Client fetches recipient pre-keys → creates session (Double Ratchet) → encrypts payload → posts to Messaging API → broker fan-out to recipients' queues → recipients decrypt & ack.

4.2.2 File Sharing

Client requests upload URL → uploads file to object storage → sends encrypted manifest (reference + hashes) → recipients download via signed URLs.

4.2.3 Channel Scheduled Post

Owner creates post with schedule → Scheduler enqueues → Messaging fan-out to subscribers.

4.2.4 Voice/Video Call

Caller sends invite → NAT traversal (ICE/TURN) → SFU mixes media → E2EE SRTP → teardown.

4.2.5 Report Content

Client submits report → Moderation stores ticket → filters classify → reviewers act.

CHAPTER 5

Deployment/Implementation View

5.1 Topology

Multi-region Kubernetes; stateless services behind load balancers; replicated DBs; object storage + CDN; secrets vault; audit logging.

5.2 Stores

- PostgreSQL per service; Redis for presence/timers; Elastic for search.
- Object storage for media; encrypted backups.

5.3 Scalability & Availability

Horizontal autoscaling; partition users by region; 99.95% uptime; blue/green deployments; circuit breakers; rate limits.

CHAPTER 6

Development View

6.1 Repository Layout

```
/clients/  
  android/, ios/, web/ (MVC/MVVM)  
/services/  
  gateway/, identity/, messaging/, channels/, media/,  
  rtc/, payments/, moderation/, analytics/, notification/  
/shared/  
  proto/, openapi/, auth/, tracing/  
/ops/  
  k8s/, helm/, terraform/, ci-cd/
```

6.2 Practices

Clean interfaces, code reviews, unit/integration tests, contract tests, linting, SAST/-DAST, and observability.

CHAPTER 7

Interface Specifications

7.1 Authentication

POST /v1/auth/start — start OTP. **POST** /v1/auth/verify — verify code, mint tokens.

7.2 Messaging

```
POST /v1/messages
Body: { toId, kind, ciphertext, mediaRef?, ttl?, viewOnce? }
GET /v1/messages/{chatId}?since=...
POST /v1/messages/{id}/ack
```

7.3 Groups & Channels

```
POST /v1/groups
POST /v1/channels
POST /v1/channels/{id}/schedule
GET /v1/channels/{id}/analytics
```

7.4 Moderation

```
POST /v1/reports
POST /v1/moderation/{userId}/strike
```

7.5 Payments

```
POST /v1/payments/transfer
POST /v1/subscriptions/{channelId}
```

CHAPTER 8

Feature Designs (by SRS Modules)

8.1 Core Communication

Messaging, media, calls, polls, search, and customization follow modular patterns.

8.2 Multi-Device

QR linking establishes trust; per-device sessions; message sync; “Saved Messages” self chat.

8.3 Privacy Tools

Disappearing messages via TTL; view-once assets; broadcast lists with private replies.

8.4 Channels & Communities

Private/public; invites; scheduling; analytics; monetization; threads; community folders.

8.5 AI Features

On-device STT; smart replies; local media tagging; meeting summaries with consent.

8.6 Moderation & Safety

Reports; automated filters; strike system; parental link with consent.

CHAPTER 9

Data Design & Schema

Entity	Key Fields
User	user_id, phone, username, profile, privacy_matrix.
Device	device_id, user_id, keys, last_seen.
Message	msg_id, chat_id, sender, ciphertext, media_ref, ttl, state.
Chat	chat_id, members, roles, policy.
Channel	channel_id, owner, settings, subscribers.
Schedule	schedule_id, payload_ref, due_ts, status.
Report	report_id, target, reason, status.
Payment	txn_id, payer, payee, amount, status.
Backup	user_id, blob_ref, created_at.

CHAPTER 10

Security & Privacy

10.1 End-to-End Encryption

- X3DH for session setup; Double Ratchet for forward secrecy.
- Groups use Sender Keys; per-device queues maintain E2EE.

10.2 Key Management

Users own backup keys; servers never see plaintext; rotations automated; compromised devices revoked.

10.3 Privacy Controls

Profile/status visibility, stealth mode, screenshot notification, download-blocking.

10.4 Hardening

TLS; OAuth2; prepared statements; CSP; rate limits; audits.

CHAPTER 11

Quality Attributes & NFR Mapping

Attribute	Design Response
Performance	Low-latency websockets; CDN; async fan-out.
Reliability	Replication; retries; idempotent ops.
Security	Signal E2EE; audits; secret vault.
Usability	Accessible UI; instant feedback.
Maintainability	Modular services; clear APIs.
Portability	Standard protocols; multi-cloud ready.
Testability	Contract tests; simulators.

CHAPTER 12

Sequence Models

12.1 Text Message Send

1. Client obtains pre-keys; establishes session.
2. Encrypts; POST /messages.
3. Service persists; emits event.
4. Recipient decrypts; ack sent.

12.2 Channel Scheduled Post

1. Admin creates post with timestamp.
2. Scheduler enqueues; Messaging fan-out.
3. Analytics updates; notifications sent.

12.3 Call Setup

1. Invite → ICE/TURN.
2. SFU negotiates; media flows.
3. End; CDR stored.

CHAPTER 13

State Models

13.1 Message Lifecycle

Draft → Queued → Sent → Delivered → Read → Expired.

13.2 Call Lifecycle

Idle → Ringing → Connected → On Hold → Ended/Failed.

13.3 Report Ticket

New → Triaged → Pending → Resolved → Sanctioned/Dismissed.

CHAPTER 14

Risks, Testing & Traceability

14.1 Key Risks

- Crypto misuse → vetted libraries & audits.
- Scalability → autoscaling, load tests.
- Abuse → moderation filters, quick appeals.

14.2 Testing Strategy

Unit, integration, contract, e2e, performance, security, chaos. Trace tests to SRS features.

14.3 SRS–Design Traceability

SRS Feature	Design Elements
Registration/Login	Identity service; OTP; device keys.
Messaging	Messaging service; E2EE sessions.
File Sharing	Media service; encrypted manifests.
Calls	RTC stack; consented recordings.
Groups/Polls	Group/Channel; poll subtype.
Channels	Scheduler; analytics; subscriptions.
Privacy Controls	TTL scheduler; stealth; view-once.
AI Features	STT/Summarizer workers.
Moderation	Reporting; filters; strikes.
Payments	Payment service; ledger; gateway.